CCP6214 - Algorithm Design & Analysis (2610)

Assignment

A. General Information

1. Assignment Weightage: **40%.**
2. Group Size: **3 – 4 students per group.** You may get permission from your tutor if your group member is less than three members.
3. Deadline: **Week 13 – 22 June 2026, by 11.59pm.** Your tutor will determine their respective submission platform.
4. Interview, Demo, Q&A: **30 minutes per group**

B. Registration of Group

1. Check with your tutor. Your tutor will evaluate your assignment.

2. You are by default not allowed to register a group with members from different lab sections to prevent your mark from getting lost accidentally, compiling hundreds from many lab sections is error-prone if cross section is allowed.

C. Task

1. Perform a comparative analysis on the following two sorting algorithms as follow:
    a) Radix Sort (Process from the rightmost character).
    b) Heap Sort (Using maxheap).

2. Perform running time analysis for Hash Table Search. The analysis shall cover the following:
    a) Theoretical analysis and experiment study.
    b) Time and space complexities.

c) The sorting and search algorithms must be implemented in standard C++ programming language.

3. Using a sorting library or searching library is not allowed.

4. Using a data structure that performs sorting or searching internally is also not allowed from the language libraries.

5. Remember not to use their built-in sorting or searching library function.

6. A group should implement functions to generate the dataset as the input to the algorithm. The requirements for the dataset are specified in section DATASET below.

7. For the experiment study:
   a) The running time captured should NOT include the time for reading input or printing output (I/O).
   b) At least 10 different input sizes should be captured for each algorithm.
   c) Each member should run all algorithms and include the results in the document.

8. Conclude the following:
   a) Findings on algorithms on the same hardware.
   b) The best sorting algorithm for the array based implementation.
   c) Compare theoretically the hash table search algorithm between the array based AVL balanced binary search tree implementation and linked list based AVL balanced binary search tree implementation.

D. Algorithm

The algorithms to be implemented are listed below. – need to check

Num	Algorithm File Name	Input in the file using multiple commented lines and one uncomment line	Output
1	radix_sort_step	*dataset_n.csv *start row (row number in csv file) *end row	A file input size n named dataset_1000_radix_sorted_step_startrow_endrow.txt listing the **sorting steps** for elements from start row to end row.

2	heap_sort_step	*dataset_n.csv *start row *end row	A file with input size n named dataset_1000_heap_sorted_step_startrow_endrow.txt listing the **sorting steps** for elements from start row to end row.
3	hash_table_search_ step	dataset filenames in csv	A file named dataset 10000000_hash_table_search_step_target.txt listing the **search path (direct or tree)** for target. Element that is searched and compared with its integer field either the target is found or not found.
4	dataset_generator	size n	A file named dataset_n.csv with **n randomized unique elements**.
5	radix_sort	dataset filenames in csv	A file named dataset _n.radix_sorted_dataset_n.csv listing all the elements from the dataset in a **sorted** order. Print the **running time**. Include a screenshot of the running time from the command prompt window for every input size and in the output files.
6	heap_sort	dataset filenames in csv	A file named heap_sort_dataset_n.csv listing all the elements from the dataset in a **sorted** order. Print the **running time**. Include a screenshot of the running time from the command prompt window for every input size and in the output files.
7	hash_table_search	dataset filenames in csv	A file named hash_table_search_dataset_n.txt listing the **running time** for **best, average, and worst cases**. A single search is too fast to be captured. Perform n searches where n is the dataset size.

E. Dataset

1. A sample dataset of 1,000 elements (dataset_1000.csv).

2. The first 7 rows are listed below. Each row has 2 record fields separated by a comma: unique, random and positive integer of 10-digit starting from 1-billion and string of 5-letter of lowercase alphabet letter.

```
1000000038,uoren
1000000009,igerk
1000000048,qouez
1000000037,sitew
1000000155,gslag
1000000197,ufnja
1000000065,rezop
```

3. A group should generate datasets that are similar to the sample dataset. The requirements for the dataset:
    a) The maximum input size of the dataset is up to 9-billion (9,000,000,000) or the running time for the maximum input size to be at least 6-hour. However, it should be large enough so that the running time of the two sorting algorithms differs by at least 60 seconds of the two different time complexities.
    b) The integers should be unique, random, positive, consist of 10 digits (example is 1,234,567,890). The integers range from 1,000,000,000 to 9,999,999,999.
    c) The elements in the datasets should be in random order before sorting.

4. Use the group leader student ID as the seed number for the random number generator function right after the main function so that different groups have different inputs. Use linear collision resolution if you know that your group has the same seed number with other group leaders.

5. The mapping from letter in the student ID to digit is as follows.

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6

   For example, the student id 243UC247CT seed number will become srand((unsigned int)2431324730U);
   or mt19937_64 srand2(24313247300;

6. Each algorithm experiment should cover 10 different input sizes.

F. Documentation

1. Your document should contain the following items:
    a) Lecture section ID, tutorial section ID, group number, group ID, group leader name, group member student IDs, group member names in alphabetical order, task percentage in 100%, and task descriptions for each group member.
    b) Every group member needs to understand all the assignment submitted answers.
    c) Your mark will be deducted out of your group total mark if you have task percentage less than hundred. Help each other in the group.

2. All items stated in the task section.

3. Citations and references in APA7 format.

4. Provide only one web link as a folder to be able to view big files one-by-one in Microsoft OneDrive such as input files and output files.

5. Provide a screenshot of computer hardware specifications (Windows / Mac / Linux) for each member.

6. Provide all the running time screenshots from the command prompt window to show that you have run each input size and in the output files.
    i. How to check PC specifications on Windows 11:
        - https://www.ccleaner.com/knowledge/how-to-check-pc-specs-on-windows-10-and-windows-11

    ii. How to check PC specification on Mac
        - https://support.apple.com/en-ca/guide/mac-help/syspr35536/mac

    iii. How to check PC specification on Linux
        - https://opensource.com/article/19/9/linux-commands-hardware-information

G. Interview, Demo and Question & Answer

1. Present according to the document contents.

2. Demonstrate your algorithms as listed in the DEMO section below. Explain the time complexities of the algorithms using your document. Put your important code part explanations in the document.

3. Every member must present at least one algorithm among dataset generator, radix sort, heap sort, or hash table search. All algorithms must be presented by the group.

4. Answer the questions presented.

5. Zero mark for the whole assignment for the absentees.

6. Zero mark for the whole assignment if a group is found to have plagiarized, provides unexplainable code, provides unexplainable text in the document, fails to submit documentation in docx file format, fails to submit proofs of running time, or shares the solution with another group, or engages in any form of cheating.

H. Demo

Perform the following for the demo.

Step Num	Algorithm	Input	Output
1	dataset_generator	*Suggest a dataset size that is not too small to see the result and does not a dozen seconds to sort. *10-million (10,000,000) is used as an example. *The tutor may specify a different size	*dataset_100000000.csv (unsorted)
2	radix_sort_step	*dataset_n.csv (example is 1000.csv *start row (tutor specifies in the code file) *end row (tutor specifies in the code file)	*dataset_1000_radix_sorted_step_startrow_endrow.txt
3	heap_sort_step	*dataset_n.csv (example is 1000.csv *start row (tutor specifies in the code file) *end row (tutor specifies in the code file)	*dataset_1000_heap_sorted_step_startrow_endrow.txt
4	hash_table_search_step	*dataset_n.csv (example is 1000.csv *a found target (tutor specifies in the code file) *a not-found target (tutor specifies in the code file)	*dataset_1000_hash_table_search_step_target.txt
5	radix_sort	*dataset_1000000.csv	*radix_sorted_dataset_1000000.csv *print the running time in file and command prompt window
6	heap_sort	*dataset_1000000.csv	*heap_sorted_dataset_1000000.csv *print the running time in file and command prompt window
7	hash_table_search	*dataset_1000000.csv	*hash_table_search_sorted_1000000.txt

			*print running times for best, average and worst cases in file and command prompt window

Sample radix_sort_step_1_7.txt

(processing from the rightmost character)

```
[1000000038/uoren, 1000000009/igerk, 1000000048/qouez,
1000000037/sitew, 1000000155/gslag, 1000000197/ufnja,
1000000065/rezop] original
[1000000155/gslag, 1000000065/rezop, 1000000037/sitew,
1000000197/ufnja, 1000000038/uoren, 1000000048/qouez,
1000000009/igerk] d=10
[1000000009/igerk, 1000000037/sitew, 1000000038/uoren,
1000000048/qouez, 1000000155/gslag, 1000000065/rezop,
1000000197/ufnja] d=9
[1000000009/igerk, 1000000037/sitew, 1000000038/uoren,
1000000048/qouez, 1000000065/rezop, 1000000155/gslag,
1000000197/ufnja] d=8
[1000000009/igerk, 1000000037/sitew, 1000000038/uoren,
1000000048/qouez, 1000000065/rezop, 1000000155/gslag,
1000000197/ufnja] d=7
[1000000009/igerk, 1000000037/sitew, 1000000038/uoren,
1000000048/qouez, 1000000065/rezop, 1000000155/gslag,
1000000197/ufnja] d=6
[1000000009/igerk, 1000000037/sitew, 1000000038/uoren,
1000000048/qouez, 1000000065/rezop, 1000000155/gslag,
1000000197/ufnja] d=5
[1000000009/igerk, 1000000037/sitew, 1000000038/uoren,
1000000048/qouez, 1000000065/rezop, 1000000155/gslag,
1000000197/ufnja] d=4
[1000000009/igerk, 1000000037/sitew, 1000000038/uoren,
1000000048/qouez, 1000000065/rezop, 1000000155/gslag,
1000000197/ufnja] d=3
[1000000009/igerk, 1000000037/sitew, 1000000038/uoren,
1000000048/qouez, 1000000065/rezop, 1000000155/gslag,
1000000197/ufnja] d=2
[1000000009/igerk, 1000000037/sitew, 1000000038/uoren,
1000000048/qouez, 1000000065/rezop, 1000000155/gslag,
1000000197/ufnja] d=1
```

Sample heap_sort_step_1_7.txt

(using maxheap)

```
[1000000197/ufnja, 1000000155/gslag, 1000000065/rezop,
1000000037/sitew, 1000000009/igerk, 1000000048/qouez,
1000000038/uoren] initial
[1000000155/gslag, 1000000038/uoren, 1000000065/rezop,
1000000037/sitew, 1000000009/igerk, 1000000048/qouez,
1000000197/ufnja] i = 6
[1000000065/rezop, 1000000038/uoren, 1000000048/qouez,
1000000037/sitew, 1000000009/igerk, 1000000155/gslag,
1000000197/ufnja] i = 5
[1000000048/qouez, 1000000038/uoren, 1000000009/igerk,
1000000037/sitew, 1000000065/rezop, 1000000155/gslag,
1000000197/ufnja] i = 4
```

```
[1000000038/uoren, 1000000037/sitew, 1000000009/igerk,
1000000048/qouez, 1000000065/rezop, 1000000155/gslag,
1000000197/ufnja] i = 3
[1000000037/sitew, 1000000009/igerk, 1000000038/uoren,
1000000048/qouez, 1000000065/rezop, 1000000155/gslag,
1000000197/ufnja] i = 2
[1000000009/igerk, 1000000037/sitew, 1000000038/uoren,
1000000048/qouez, 1000000065/rezop, 1000000155/gslag,
1000000197/ufnja] i = 1
```

Sample radix_sorted_1000.csv and sample heap_sorted_1000.csv (only the first 5 rows are shown here, your file should have all rows based on the dataset size)

```
1000875538/ilihe
1001659492/ziuuj
1002487583/odtua
1002558672/iyavo
1003558672/iyavb
```

Sample hash_table_search_step_2008864030.txt (target found)

```
2008864030 = 2008864030/rdiea
```

Sample hash_table_search_step_123456789.txt (target not found)

```
-1 != 123456789
```

Sample hash_table_search_dataset_1000.txt (x, y, z are numbers)

Best case time: x seconds
Average case time: y seconds
Worst case time: z seconds

I. Submission Format

1. Check with your tutor for the submission details. The submission shall include the following items.
    a) The document in docx.
    b) The code files in cpp.
    c) The dataset input files in csv.
    d) The output files in txt.

2. There is no need to submit all the dataset input files and all the output files if the overall zipped file is over 99MB. In the document, you need to provide a web link in Microsoft Onedrive to your big files.

3. Each group is required to submit a single assignment in zip file format. Example of zip filename is T13L_G04.zip.

J. Assessment Criteria

No	Criteria	0 – No attempt	1 – Very poor	2 – Poor	3 – Moderate	4 – Good	5 – Excellent
1	Dataset generation (group)	No implementation or hard-coded data	Generate integers only or elements not randomized	Generate (integer, string) Elements randomized Integers are not unique or Generate (string, integer)	Generate (integer, string) Elements are randomized Integers are unique Integer range is < 10-million or smaller than the max input size runtime < 6-hour for the algorithm using this input	Generate (integer, string) Elements are randomized Integers are unique Integer range is >= 10-million or smaller than the max input size runtime >= 6-hour for the algorithm using this input Minor issue	Generate (integer, string) Elements are randomized Integers are unique Integer range is >= 10-million or smaller than the max input size runtime >= 6-hour for the algorithm using this input All instructions are followed, without issue
2	Radix sort from rightmost character (group)	No complexity analysis and implementation	Wrong complexity analysis and implementation or Bad filename or Bad documentation	Incomplete complexity analysis and incomplete output in document or Implementation sorts elements by string	Incomplete complexity analysis or output in document Implementation sorts elements by integer Output file has integer only	Complete complexity analysis and demo Implementation sorts elements by integer Output file has (integer, string) rows with integer sorted Minor issue	Complete complexity analysis and demo Implementation sorts elements by integer Output file has (integer, string) rows with integer sorted All instructions are followed, without issue
3	Heap sort using maxheap (group)	No complexity analysis and implementation	Wrong complexity analysis and implementation or Bad filename or Bad documentation	Incomplete complexity analysis and incomplete output in document or Implementation sorts elements by string	Incomplete complexity analysis or output in document Implementation sorts elements by integer Output file has integer only	Complete complexity analysis and demo Implementation sorts elements by integer Output file has (integer, string) rows with integer sorted Minor issue	Complete complexity analysis and demo Implementation sorts elements by integer Output file has (integer, string) rows with integer sorted All instructions are followed without issue

4	Hash table search (group)	No complexity analysis and implementation	Wrong complexity analysis and implementation or Bad filename or Bad documentation	Incomplete complexity analysis and incomplete output in document or Implementation search element by string	Incomplete complexity analysis or output in document Implementation search elements by integer Output file has integer only	Complete complexity analysis and demo Implementation search elements by integer Output file has (integer, string) rows with running times of three cases Minor issue	Complete complexity analysis and demo Implementation search elements by integer Output file has (integer, string) rows with running times of three cases All instructions are followed, without issue
5	Conclusion (group)	No conclusion and AVL comparison by theory	Poor conclusion, not supported by analysis and experiments Poor AVL comparison by theory	Moderate conclusion, somewhat supported by analysis and experiments Poor AVL comparison by theory	Moderate conclusion, somewhat supported by analysis and experiments Moderate AVL comparison by theory	Good conclusion, strongly supported by analysis and experiments Good AVL comparison by theory	Excellent conclusion, strongly supported by comprehensive analysis and experiments Excellent AVL comparison by theory
6	Document clarity and completeness (group)	No docx	< 50% required contents	50% to 80% required contents	Include >= 80% contents without citation and references	Complete contents, citation and references with minor issue	Clear and complete contents, citation and references without issue Provided complete documentation in the docx file, web link to all big files in folder form, running time proofs, hardware specs

7	Experiments (individual)	No experiment result	≤ 5 input files or no chart, table only	< 10 input file, table only	≥ 10 input files The running time of the algorithms in the largest dataset size are separated by less than 1 second Cover all sorting and searching algorithms in document No code file, no input files, no output files, no big dataset input file link, no big output file link, no running time proofs, or hardware spec is not provided	≥ 10 input files The running time of the algorithms in the largest dataset size are separated by < 60 seconds Cover all sorting and searching algorithms in document	≥ 10 input files per algorithm The running time of the algorithms in the largest dataset size are separated by >= 60 seconds Cover all sorting and searching algorithms Provided complete experiment documentation in the docx file, code files, input files, output files, web link to all big files in folder form, running time proofs
8	Interview, demo and Q&A (individual)	0 mark for the whole assignment if absent or no interview	Reading docx/note, unable to explain code or answer questions	Poor in interview Barely able to explain code or answer questions	Moderate in interview, demo and Q&A	Good in interview, demo and Q&A	Excellent in interview, demo and Q&A

K. Assessment Criteria

Group member names in alphabetically order. Label group leader in parentheses.

1. Student Name A
2. Student Name B if this is the group leader (leader)
3. Student Name C
4. Student Name D

Questions	Criteria	Weight	Actual Mark
Q1	Dataset generation (group)	5	?
Q2	Radix sort (group)	5	?
Q3	Heap sort (group)	5	?
Q4	Hash table search (group)	5	?

Q5	Conclusion (group)	5	?
Q6	Document clarity and completeness (group)	5	?
Q7	Experiments (individual)	5	[?] [?] [?] [?]
Q8	Interview, demo and Q&A (individual)	5	[?] [?] [?] [?]
	Total mark	40	[?] [?] [?] [?]

CCP6214 Algorithm Design and Analysis (2610)
Assignment (40 marks)

Lecture section: ?TC4L
Tutorial section: ?T13L
Group number: ?G04
Group ID: ?T13L_G04

Group leader name: ?

No	Student ID	Student name by alphabetical order and phone number, label group leader in parentheses	Task percentage in 100%
1			
2			
3			
4			

**Every student is responsible for 100% (task percentage) of this group assignment work.

Marksheet checklist (40%) // on new page

Assignment programming, documentation and interview

- You are required to submit assignment to your respective tutor before the submission deadline.
- Also document all your assignment tasks with this marking table that contain cover page, table of contents, page numbering, inputs, outputs, screenshots, explanations, and others. Group member names in alphabetically order. Label group leader in parentheses.
    1. Student Name A
    2. Student Name B if this is the group leader (leader)
    3. Student Name C
    4. Student Name D

Questions	Criteria	Weight	Actual Mark
Q1	Dataset generation (group)	5	?
Q2	Radix sort (group)	5	?
Q3	Heap sort (group)	5	?
Q4	Hash table search (group)	5	?
Q5	Conclusion (group)	5	?
Q6	Document clarity and completeness (group)	5	?
Q7	Experiments (individual)	5	[?] [?] [?] [?]
Q8	Interview, demo and Q&A (individual)	5	[?] [?] [?] [?]
	Total mark	40	[?] [?] [?] [?]

Additional comments:

You are required to fill in your task percentage and task descriptions.
Every student is responsible for 100% (task percentage) of this group assignment work.

Student 1

Student ID	?
Student name	?
Task percentage	?
Task descriptions	?

Student 2

Student ID	?
Student name	?
Task percentage	?
Task descriptions	?

Student 3

Student ID	?
Student name	?
Task percentage	?
Task descriptions	?

Student 4

Student ID	?
Student name	?
Task percentage	?
Task descriptions	?

Each feature will be evaluated based on documentation, fulfilment of requirements, correctness, compilation without warnings and errors, error free during runtime, error handlings, quality of comments, user friendliness, good coding format and style.

Table of contents with page numbers and links // on new page

?

Insert the comment below at the beginning of your source code files // on new page

```
// *********************************************************
// Program: YOUR_FILENAME.cpp
// Course: CCP6214 Algorithm Design and Analysis
// Lecture Class: TC4L
// Tutorial Class: T13L
// Trimester: 2610
// Member_1: ID | NAME | EMAIL | PHONE
// Member_2: ID | NAME | EMAIL | PHONE
// Member_3: ID | NAME | EMAIL | PHONE
// Member_4: ID | NAME | EMAIL | PHONE
// *********************************************************
// Task Distribution
// Member_1:
// Member_2:
// Member_3:
// Member_4:
// *********************************************************
```

Question Section // on new page

Q1 [5] Question title

Question tasks

?

Screenshots, inputs, outputs, figures, explanations, tables, charts

?

Code parts, explanations

?

. . . .

. . . .

. . . .

. . . .

. . . .

. . . .

Q2 [5] Question title // on new page

Question tasks

?

Screenshots, inputs, outputs, figures, explanations, tables, charts

?

Code parts, explanations

?

Citations and references section // on new page

?